
**PINNs for Laplace's Equation:
Complex Analysis and AI for Science**

An Independent Project Report

Chelsie Tao

July 1, 2026

Abstract

This report documents an independently developed project that connects two areas of study: complex analysis and scientific machine learning. The project is built around a simple but structurally complete example, the analytic function $f(z) = z^2$, and asks whether a neural network can learn to satisfy a partial differential equation (PDE) not from labeled interior data, but from the differential equation itself together with boundary information. A Physics-Informed Neural Network (PINN) is implemented in PyTorch to solve the Dirichlet boundary value problem for Laplace's equation on the square domain $D = [-1, 1]^2$, using the harmonic function $u(x, y) = x^2 - y^2$, the real part of $f(z) = z^2$, as the analytic benchmark against which the learned solution is evaluated. The project proceeds from the mathematical foundations, namely the Cauchy-Riemann equations and the derivation of Laplace's equation for harmonic functions, through the construction and training of the PINN, to error analysis, harmonic conjugate recovery, complex-function visualization via domain coloring, and a physical interpretation of the result in terms of two-dimensional potential flow. The work is intended to demonstrate, in a self-contained and reproducible form, how classical results from complex analysis can be reformulated as a computational and learning problem, and to serve as evidence of mathematical understanding, programming ability, and interdisciplinary research potential in the area broadly known as AI for Science.

Keywords: Physics-Informed Neural Networks, Laplace's equation, Cauchy-Riemann equations, harmonic functions, complex analysis, automatic differentiation, scientific machine learning.

Contents

1	Project Overview and Positioning	3
1.1	Project Resources	3
2	Core Research Question	4
3	Mathematical Foundations	4
3.1	The Cauchy-Riemann Equations	4
3.2	Laplace's Equation and Harmonic Functions	5
4	The PINN Problem Formulation	5
5	Methodology and Implementation	6
5.1	Tools	6
5.2	Network Architecture	6
5.3	Loss Function	7
5.4	Automatic Differentiation	8
5.5	Training Convergence	8
6	Results	9
7	Project Structure	15
8	Connections to the Complex Analysis Coursework	16
9	Discussion	17
10	Future Directions	17
11	Conclusion	18
	Appendix: Project Summary for External Use	18
	References	18

1 Project Overview and Positioning

This project, *PINNs for Laplace's Equation: Complex Analysis and AI for Science*, was developed independently after completing coursework in complex analysis, as a way of extending the classroom material into a computational, trainable, and visualizable research artifact. The central idea is to take the most basic but structurally complete object in complex analysis, an analytic function, and treat the partial differential equation that its real part satisfies as a learning target for a neural network.

In one sentence, the project converts the classical theory of analytic functions, the Cauchy-Riemann equations, harmonic functions, and Laplace's equation, into a problem that can be computed, trained, and visualized using modern deep learning tools. It is explicitly framed here as a self-directed project rather than a course assignment or formal final report; the report format is used only because it provides a clear and complete way of presenting the motivation, theory, implementation, and results in one document.

1.1 Project Resources

The project is hosted publicly and organized as follows.

- **GitHub repository:** <https://github.com/chelsietao/pinn-laplace-complex-analysis>, containing the README, source code (src), static assets (public/assets), a continuous integration workflow, and the Vite-based project configuration used to build the accompanying website. The README is titled *PINNs for Laplace's Equation / Complex Analysis $\times$ AI for Science Learning Notes*.
- **Project website (GitHub Pages):** <https://chelsietao.github.io/pinn-laplace-complex-analysis/>, an interactive presentation of the project results.
- **Core notebook:** `PINN_Laplace_ComplexAnalysis_v2.ipynb`, containing the full implementation: theoretical setup, model definition, training loop, and visualization code.
- **Reference lecture notes:** *Lecture notes on complex analysis: Analytic functions, integrations and singularities*, by Teh, Jyh-Haur, Department of Mathematics, National Tsing Hua University, covering complex differentiation, analytic functions, the Cauchy integral formula, the residue theorem, Rouché's theorem, and the Riemann mapping theorem.
- **Generated figures:** a set of nine output images produced by the notebook, summarized in Table 1 and referenced throughout this report.

Table 1: Figures produced by the project notebook.

File	Content
<code>section2_analytic_solution.png</code>	Visualization of $u(x,y) = x^2 - y^2$ and $v(x,y) = 2xy$
<code>section3_training_points.png</code>	Interior and boundary collocation points
<code>section3_training_curves.png</code>	PDE loss and boundary loss during training
<code>section4_error_analysis.png</code>	Error map between PINN prediction and exact solution
<code>section4_cross_section.png</code>	One-dimensional cross-sections of predicted vs. exact u
<code>section5_harmonic_conjugate.png</code>	Recovered harmonic conjugate v
<code>section6_domain_coloring.png</code>	Domain coloring of the complex map $f(z) = z^2$
<code>section7_fluid_flow.png</code>	Equipotential lines and streamlines (potential flow analogy)
<code>final_summary.png</code>	Combined summary figure

2 Core Research Question

The question that motivates this project can be stated as follows:

Can a neural network learn a function that satisfies Laplace's equation, not by fitting labeled data points throughout the domain, but by being constrained directly by the differential equation itself and by the values prescribed on the boundary of the domain?

To answer this question in a setting where the ground truth is known and easy to verify, the project starts from one of the simplest nontrivial analytic functions,

$$f(z) = z^2. \quad (1)$$

Writing $z = x + iy$ with $x, y \in \mathbb{R}$ gives

$$f(z) = (x + iy)^2 = x^2 - y^2 + i(2xy), \quad (2)$$

so that the real and imaginary parts of f are, respectively,

$$u(x, y) = x^2 - y^2, \quad v(x, y) = 2xy. \quad (3)$$

Since $f(z) = z^2$ is analytic on all of $\mathbb{C}$, classical complex analysis guarantees that both u and v are harmonic functions, that is, they satisfy Laplace's equation, and that v is the harmonic conjugate of u . This pair (u, v) therefore provides a closed-form, fully known benchmark solution against which a numerically learned solution can be compared exactly, which is precisely what makes it a convenient starting point for testing a PINN.

3 Mathematical Foundations

3.1 The Cauchy-Riemann Equations

A complex-valued function $f(z) = u(x, y) + iv(x, y)$, viewed as a map from a subset of $\mathbb{R}^2$ to $\mathbb{R}^2$, is analytic (complex differentiable) at a point if and only if its real and imaginary parts satisfy the Cauchy-Riemann equations:

$$u_x = v_y, \quad (4)$$

$$u_y = -v_x. \quad (5)$$

For the example used in this project,

$$u(x, y) = x^2 - y^2, \quad v(x, y) = 2xy, \quad (6)$$

direct computation gives

$$u_x = 2x, \quad u_y = -2y, \quad v_x = 2y, \quad v_y = 2x. \quad (7)$$

Comparing these expressions confirms that

$$u_x = v_y = 2x, \quad u_y = -v_x = -2y, \quad (8)$$

so the Cauchy-Riemann equations are satisfied everywhere in $\mathbb{R}^2$, consistent with $f(z) = z^2$ being entire (analytic on the whole complex plane).

3.2 Laplace's Equation and Harmonic Functions

Definition 1. A twice continuously differentiable function $u : D \subset \mathbb{R}^2 \rightarrow \mathbb{R}$ is said to be harmonic on D if it satisfies Laplace's equation,

$$\Delta u = u_{xx} + u_{yy} = 0 \quad \text{in } D. \quad (9)$$

A standard consequence of the Cauchy-Riemann equations, obtained by differentiating $u_x = v_y$ with respect to x and $u_y = -v_x$ with respect to y and adding the results (using the equality of mixed partials, i.e. Clairaut's theorem, for sufficiently smooth functions), is that the real and imaginary parts of any analytic function are automatically harmonic. This project verifies the statement directly for the chosen example. For

$$u(x, y) = x^2 - y^2, \quad (10)$$

the second-order partial derivatives are

$$u_{xx} = 2, \quad u_{yy} = -2, \quad (11)$$

so that

$$u_{xx} + u_{yy} = 2 + (-2) = 0. \quad (12)$$

Hence u is harmonic, and this fact, derived here from elementary calculus, is exactly the property that the PINN described in Section 4 is asked to rediscover numerically, without being given the closed-form expression of u in the interior of the domain.

As a sanity check, the notebook also verifies this identity numerically rather than only symbolically: $u(x, y) = x^2 - y^2$ is evaluated on a dense 300×300 grid on $[-2, 2]^2$, and $u_{xx} + u_{yy}$ is approximated at every grid point with a standard second-order finite-difference stencil. The resulting maximum absolute value of the discrete Laplacian over the interior of the grid is 1.99×10^{-11} , which is zero up to floating-point and discretization error. This confirms that the closed-form benchmark used throughout the rest of the project is harmonic not only in theory but also under the same kind of numerical evaluation the PINN itself will be subjected to.

4 The PINN Problem Formulation

The numerical problem solved in this project is a Dirichlet boundary value problem for Laplace's equation on the square domain

$$D = [-1, 1] \times [-1, 1] \subset \mathbb{R}^2. \quad (13)$$

The governing equation and boundary condition are

$$\nabla^2 u(x, y) = 0 \quad \text{for } (x, y) \in D, \quad (14)$$

$$u(x, y) = x^2 - y^2 \quad \text{for } (x, y) \in \partial D, \quad (15)$$

where ∂D denotes the boundary of the square. Because $u(x, y) = x^2 - y^2$ is itself harmonic everywhere on D and matches the prescribed boundary data exactly, it is also the unique analytic solution of the boundary value problem by the maximum principle for harmonic functions (uniqueness of solutions to the Dirichlet problem). This gives a closed-form ground truth,

$$u_{\text{exact}}(x, y) = x^2 - y^2, \quad (16)$$

against which the network's prediction can be compared at every point of D , not only on a held-out test set.

The task given to the neural network is deliberately restricted: the model is never shown the value of u at interior points of D . It is trained using only two sources of information,

- (1) the requirement that the PDE residual, $u_{xx} + u_{yy}$, vanish at a set of randomly sampled interior collocation points, and
- (2) the requirement that the network's output match the known boundary values at a set of points sampled on ∂D .

From these two constraints alone, the network must recover the full two-dimensional field $u(x, y)$ throughout the interior of the domain.

5 Methodology and Implementation

5.1 Tools

The implementation relies on a standard scientific Python stack:

- **Python** as the implementation language;
- **PyTorch** (version 2.11.0, CUDA 12.8 build, as recorded by the notebook at run time), used both to define the neural network and, critically, to compute the derivatives required by the PDE residual through its automatic differentiation engine;
- **NumPy** (version 2.0.2) for numerical array operations and sampling of collocation points;
- **Matplotlib** for all static visualizations, including the analytic solution plots, training curves, error maps, and domain coloring;
- **Google Colab**, running on an NVIDIA Tesla T4 GPU with 15.6 GB of VRAM, as the runtime environment for training.

For reproducibility, the random seeds for both PyTorch and NumPy were fixed (`torch.manual_seed(42)`, `np.random.seed(42)`) before model initialization and data sampling.

5.2 Network Architecture

The PINN is a fully connected feedforward neural network that maps spatial coordinates to a scalar prediction of the solution,

$$(x, y) \mapsto \hat{u}_\theta(x, y), \quad (17)$$

where θ denotes the collection of trainable weights and biases. Concretely, the network used in the notebook has the layer sequence

$$2 \rightarrow 64 \rightarrow 64 \rightarrow 64 \rightarrow 64 \rightarrow 1, \quad (18)$$

i.e. a two-unit input layer for (x, y) , four hidden layers of 64 units each, and a single linear output unit for $\hat{u}_\theta(x, y)$. This gives a total of 12,737 trainable parameters, as confirmed by the notebook's own parameter count at model-construction time. All weight matrices are initialized with Xavier (Glorot) normal initialization and all biases are initialized to zero, a choice noted in the implementation as important for stabilizing PINN training, since poorly scaled initial weights can make the second-derivative terms in the PDE residual numerically unstable early in training.

The hidden layers use the hyperbolic tangent ($\tanh$) activation function rather than a non-smooth activation such as ReLU. This choice is motivated directly by the structure of the loss function: because the training objective requires differentiating the network output twice with respect to its spatial inputs, the activation function must itself be smooth (infinitely differentiable) so that these second derivatives are well

defined and numerically stable. $\tanh$ satisfies this requirement, whereas a piecewise-linear activation such as ReLU has a vanishing or undefined second derivative almost everywhere, making it unsuitable for PINNs that rely on second-order PDE residuals.

5.3 Loss Function

The network is trained by minimizing a composite loss function that combines the PDE residual and the boundary condition mismatch,

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{PDE}} + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}}, \quad (19)$$

where λ_{BC} is a scalar weight balancing the relative importance of the two terms. The PDE residual loss is evaluated at N interior collocation points $\{(x_i, y_i)\}_{i=1}^N$,

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N} \sum_{i=1}^N \left(\hat{u}_{xx}(x_i, y_i) + \hat{u}_{yy}(x_i, y_i) \right)^2, \quad (20)$$

and the boundary condition loss is evaluated at M boundary points $\{(x_j, y_j)\}_{j=1}^M$ sampled on ∂D ,

$$\mathcal{L}_{\text{BC}} = \frac{1}{M} \sum_{j=1}^M \left(\hat{u}_{\theta}(x_j, y_j) - u_{\text{exact}}(x_j, y_j) \right)^2. \quad (21)$$

Minimizing $\mathcal{L}_{\text{total}}$ jointly drives the network toward a function that is approximately harmonic in the interior of D and that matches the prescribed values of $u(x, y) = x^2 - y^2$ on ∂D , which together are sufficient, by the theory of the Dirichlet problem, to pin down the solution throughout the domain.

In the actual training loop, the two point sets are treated asymmetrically. The boundary set is fixed once, with $M = 800$ points distributed evenly over the four edges of the square (200 points per edge), sampled before training begins. The interior set, by contrast, is *resampled fresh at every epoch*: $N = 2,000$ new interior points are drawn uniformly from D at the start of each training step, so that over the course of training the PDE residual is enforced on a continually changing cloud of collocation points rather than a single fixed grid. (A separate, larger batch of 3,000 interior and 800 boundary points is also generated once, purely to produce the collocation-point visualization in Figure 2; this larger batch is not the one used directly inside the training loop itself.)

Optimization uses the Adam optimizer with an initial learning rate of 10^{-3} , decayed by a factor of 0.5 every 2,000 epochs via a step learning-rate scheduler, for a total of 8,000 epochs. The boundary-condition weight is set to $\lambda_{\text{BC}} = 10.0$, placing more relative emphasis on satisfying the boundary data than on the interior residual at any single step. After each backward pass, the gradient norm is clipped to a maximum of 1.0 to prevent the second-order derivative terms in $\mathcal{L}_{\text{PDE}}$ from producing exploding gradients early in training. Table 2 summarizes these settings.

Table 2: Training hyperparameters used in the notebook.

Hyperparameter	Value
Network layers	$2 \rightarrow 64 \rightarrow 64 \rightarrow 64 \rightarrow 64 \rightarrow 1$
Trainable parameters	12,737
Activation	tanh (hidden layers)
Weight / bias initialization	Xavier normal / zero
Optimizer	Adam
Initial learning rate	1×10^{-3}
LR schedule	$\times 0.5$ every 2,000 epochs (StepLR)
Epochs	8,000
λ_{BC}	10.0
Gradient clipping	max norm 1.0
Interior points N	2,000 (resampled every epoch)
Boundary points M	800 (fixed, 200 per edge)

5.4 Automatic Differentiation

The first- and second-order spatial derivatives required for the PDE residual, namely u_x , u_y , u_{xx} , and u_{yy} , are computed using PyTorch's autograd system rather than finite-difference approximation. The function `torch.autograd.grad` is used to differentiate the scalar network output with respect to its input coordinates, and is applied a second time to the resulting first derivatives to obtain the second-order terms needed for the Laplacian. This is the mechanism that allows the differential equation itself, rather than any precomputed derivative data, to enter the training loss.

A simplified illustration of the pattern used to compute the Laplacian of the network output is shown below.

Listing 1: Simplified pattern for computing the PDE residual using automatic differentiation.

```

1 def pde_residual(model, x, y):
2     x.requires_grad_(True)
3     y.requires_grad_(True)
4     u = model(torch.cat([x, y], dim=1))
5
6     u_x = torch.autograd.grad(u, x, grad_outputs=torch.ones_like(u),
7                               create_graph=True)[0]
8     u_y = torch.autograd.grad(u, y, grad_outputs=torch.ones_like(u),
9                               create_graph=True)[0]
10    u_xx = torch.autograd.grad(u_x, x, grad_outputs=torch.ones_like(u_x),
11                               create_graph=True)[0]
12    u_yy = torch.autograd.grad(u_y, y, grad_outputs=torch.ones_like(u_y),
13                               create_graph=True)[0]
14
15    return u_xx + u_yy  # should approach 0 for a harmonic solution

```

5.5 Training Convergence

Table 3 reproduces the training log recorded by the notebook, printed every 500 epochs over the full 8,000-epoch run. The total loss drops by roughly two orders of magnitude within the first 2,500 epochs, with a further sharp drop visible immediately after the first learning-rate decay step at epoch 2,000; both the PDE

residual loss and the boundary-condition loss continue to decrease essentially monotonically thereafter, reaching $\mathcal{L}_{\text{PDE}} \approx 1.3 \times 10^{-5}$ and $\mathcal{L}_{\text{BC}} \approx 0$ (to six decimal places) by epoch 8,000. This behavior is also shown graphically in Figure 2 (right panel).

Table 3: Training log recorded by the notebook (loss values every 500 epochs).

Epoch	Total Loss	PDE Loss	BC Loss
500	0.006101	0.000613	0.000549
1000	0.005363	0.000443	0.000492
1500	0.004729	0.000379	0.000435
2000	0.005254	0.000512	0.000474
2500	0.000051	0.000044	0.000001
3000	0.000633	0.000075	0.000056
3500	0.000218	0.000039	0.000018
4000	0.001205	0.000129	0.000108
4500	0.000027	0.000025	0.000000
5000	0.000026	0.000024	0.000000
5500	0.000020	0.000019	0.000000
6000	0.000024	0.000019	0.000000
6500	0.000018	0.000017	0.000000
7000	0.000016	0.000015	0.000000
7500	0.000016	0.000015	0.000000
8000	0.000014	0.000013	0.000000

6 Results

The completed project produced the following concrete outcomes, each corresponding to one stage of the notebook and, where applicable, to one of the figures listed in Table 1.

1. Derivation of the real and imaginary parts of $f(z) = z^2$, giving $u(x, y) = x^2 - y^2$ and $v(x, y) = 2xy$.
2. Direct verification that u and v satisfy the Cauchy-Riemann equations.
3. Direct verification, both symbolically and via a finite-difference check on a 300×300 grid (maximum discrete Laplacian 1.99×10^{-11}), that $u(x, y) = x^2 - y^2$ satisfies Laplace's equation.
4. Construction of a PINN (12,737 trainable parameters; see Table 2) that solves the Dirichlet problem for Laplace's equation on $D = [-1, 1]^2$.
5. Training of the model for 8,000 epochs using only the PDE residual loss on interior collocation points and the boundary condition loss on boundary points, without access to interior ground-truth values; the total loss falls from approximately 6×10^{-3} to 1.4×10^{-5} over training (Table 3).
6. Quantitative comparison between the PINN prediction and the exact analytic solution on a dense 200×200 evaluation grid, yielding an L_2 error of 0.000215, a maximum pointwise error of 0.001148, and a mean absolute error of 0.000193, visualized as a two-dimensional error map and as one-dimensional cross-sections (Figure 3).

7. Recovery of the harmonic conjugate v from the network's learned values of u , by numerically integrating u_x (obtained via autograd) along the y -direction using the Cauchy-Riemann relation $v_y = u_x$; the resulting field satisfies the Cauchy-Riemann check $u_x = v_y$ with a mean absolute discrepancy of essentially 0 (Figure 4).
8. Visualization of complex maps using domain coloring (hue encoding $\arg(f(z))$, brightness encoding $|f(z)|$), applied not only to $f(z) = z^2$ but to a small gallery of six functions, z^2 , z^3 , e^z , $1/z$, $\sin(z)$, and $(z^2 - 1)/(z^2 + 1)$, illustrating how zeros and poles appear as counterclockwise and clockwise hue cycles, respectively, with the cycle count matching the order of the zero or pole (Figure 5).
9. An extended physical interpretation of the harmonic pair (u, v) as the velocity potential and stream function of an idealized two-dimensional fluid flow, identifying $f(z) = z^2$ specifically with ideal corner flow around a right-angle bend, with a stagnation point at the origin and flow symmetric about both axes (Figure 6).
10. Construction of an interactive GitHub Pages website presenting the project results.

Figures 1–7 reproduce the visual outputs of the notebook referenced above. (Figure files should be placed in the same directory as this document, or the graphics path updated accordingly, for the references below to resolve when compiling.)

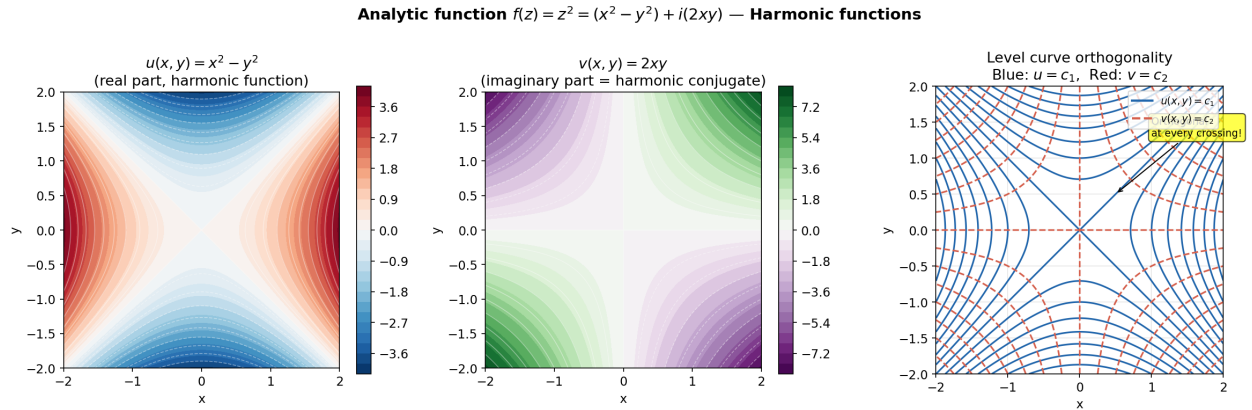


Figure 1: Analytic solution: $u(x, y) = x^2 - y^2$ and its harmonic conjugate $v(x, y) = 2xy$.

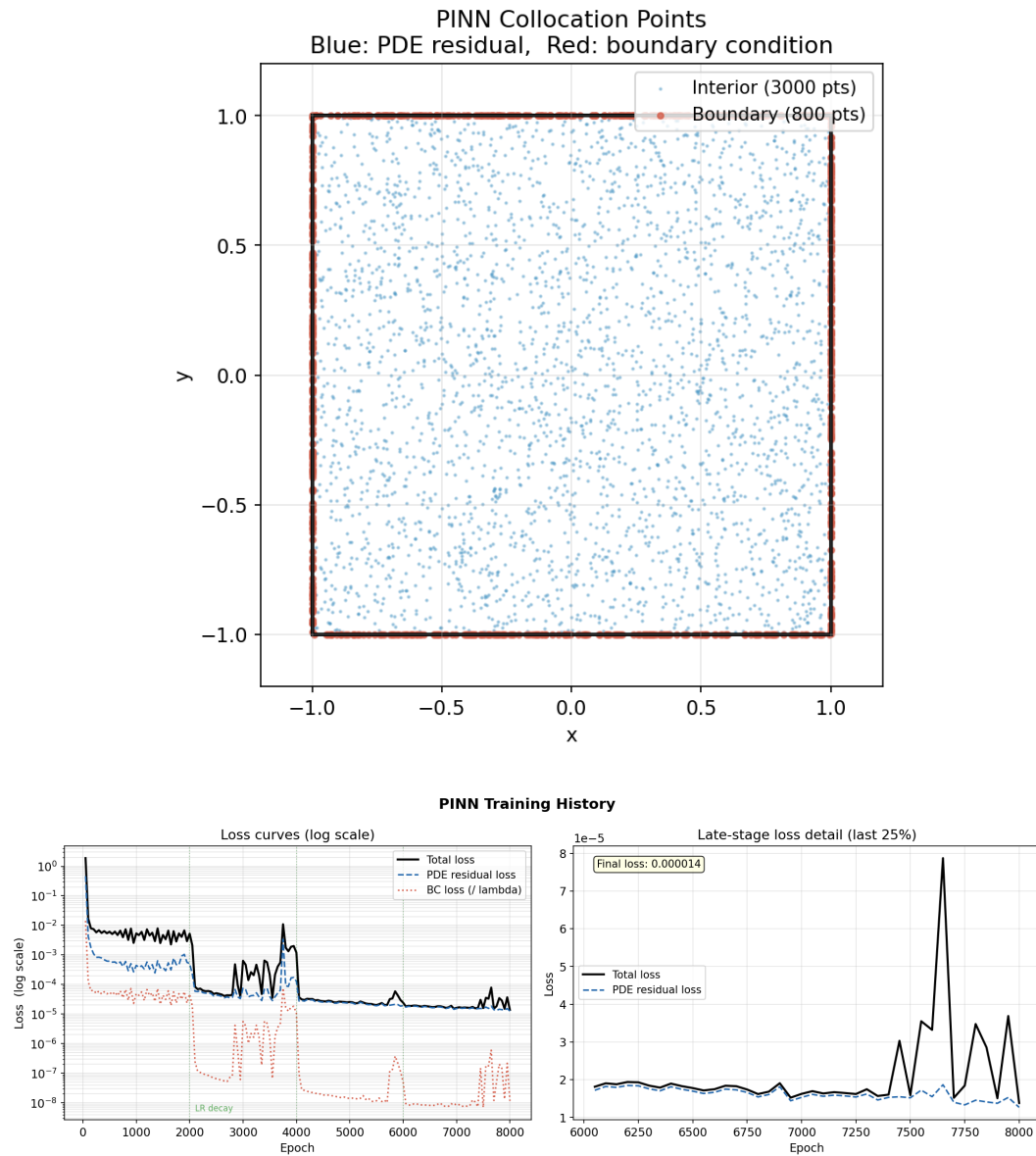


Figure 2: Top: interior and boundary collocation points used for training. Bottom: evolution of the PDE residual loss and boundary loss over training.

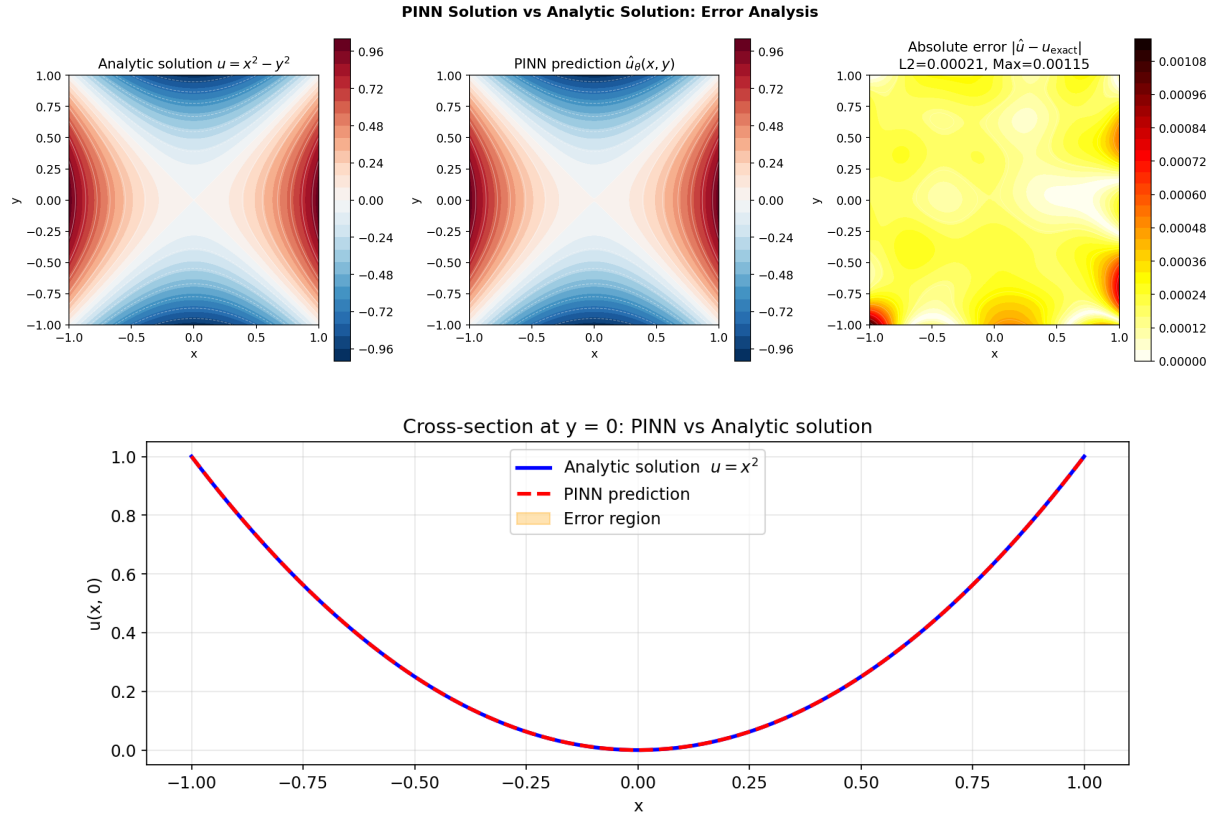


Figure 3: Top: error map between the PINN prediction and the exact solution over D . Bottom: cross-sections comparing predicted and exact values of u .

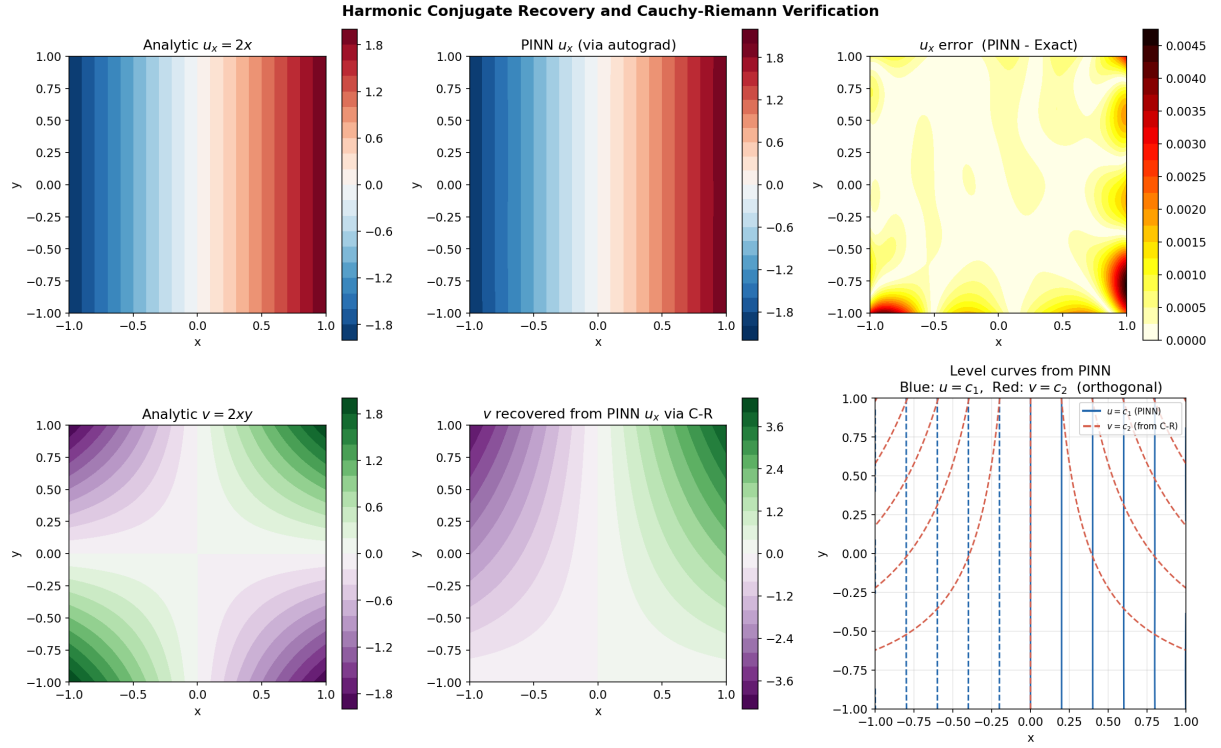


Figure 4: Harmonic conjugate v recovered from the PINN's learned values of u via the Cauchy-Riemann equations.

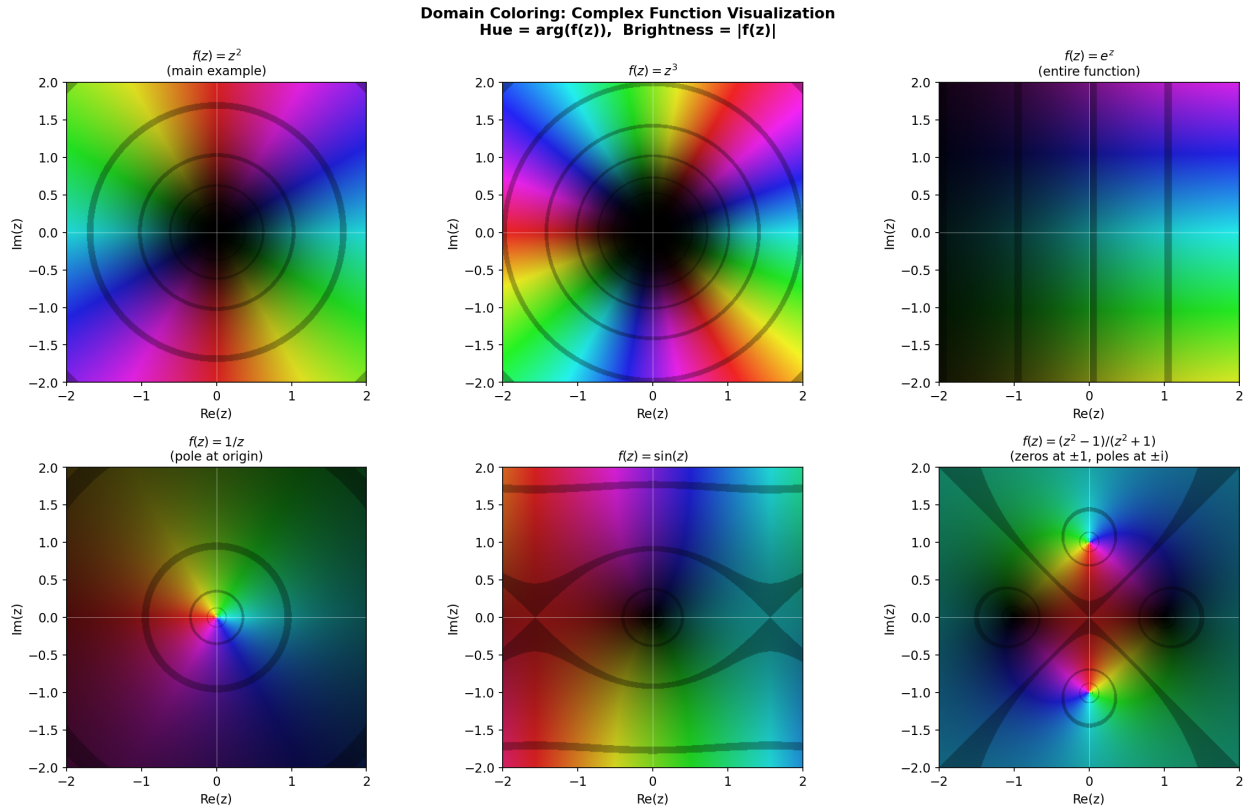


Figure 5: Domain coloring visualization gallery: hue encodes $\arg(f(z))$, brightness encodes $|f(z)|$, shown for $f(z) = z^2$ (the project's main example) alongside z^3 , e^z , $1/z$, $\sin(z)$, and $(z^2 - 1)/(z^2 + 1)$.

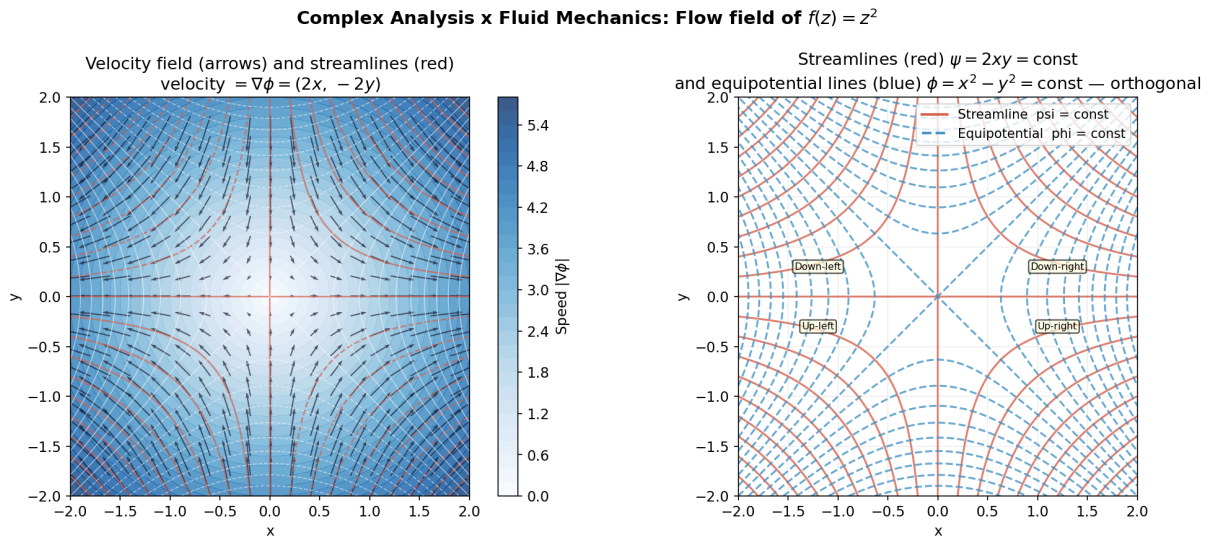


Figure 6: Potential flow interpretation: equipotential lines of u and streamlines of v , shown to be mutually orthogonal. $f(z) = z^2$ corresponds to ideal flow around a 90° corner, with a stagnation point at the origin.

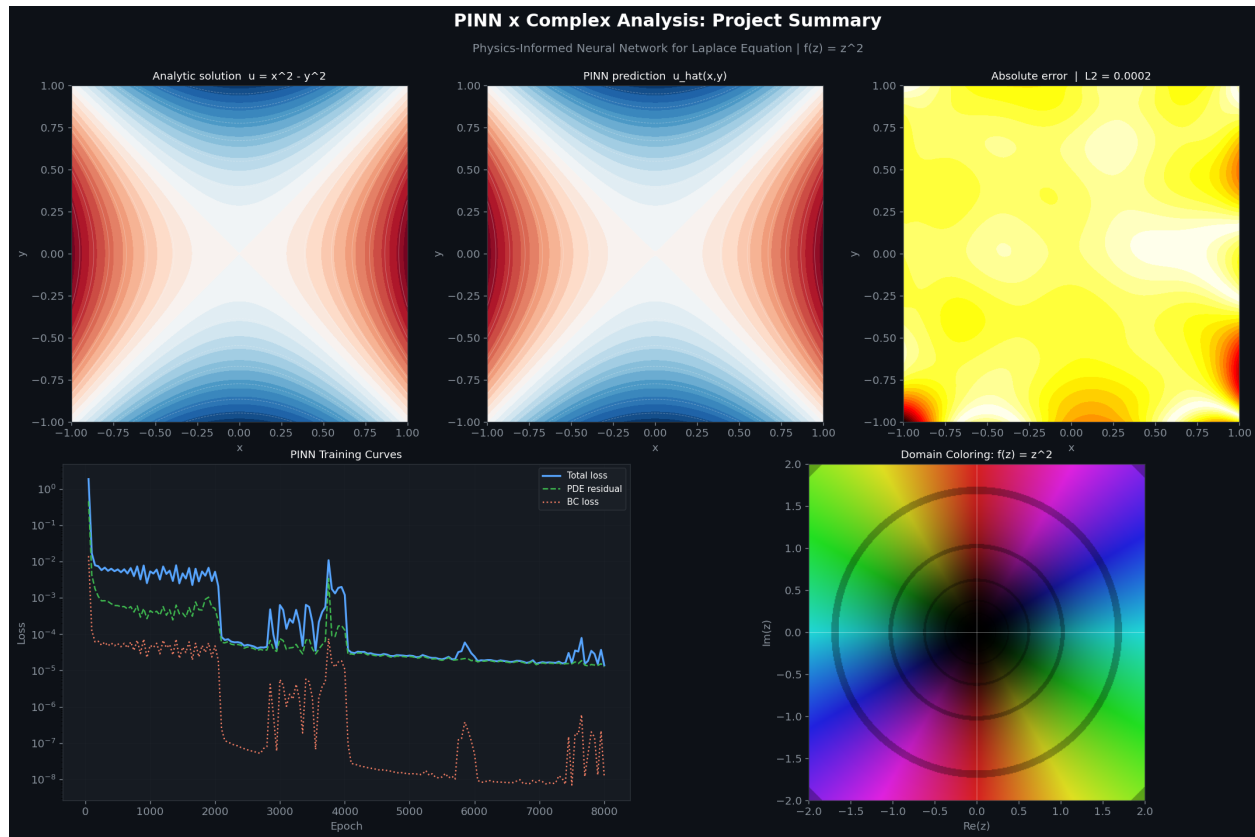


Figure 7: Combined summary dashboard: analytic solution, PINN prediction, absolute error map, training loss curves, and domain coloring of $f(z) = z^2$, assembled into a single five-panel figure.

7 Project Structure

The accompanying notebook, `PINN_Laplace_ComplexAnalysis_v2.ipynb`, is organized into eight sections, summarized in Table 4; the notebook's own header estimates a total run time of roughly 25–35 minutes, including reading the theoretical material.

Table 4: Section-by-section structure of the project notebook.

Section	Content	Time
Section 0	Theoretical review: the Cauchy-Riemann equations, harmonic functions, Laplace's equation, and the core idea of physics-informed learning.	10 min (read)
Section 1	Environment setup: PyTorch, NumPy, Matplotlib, and GPU availability check.	2 min
Section 2	Visualization of the analytic solution: plotting $u(x, y) = x^2 - y^2$ and $v(x, y) = 2xy$ and verifying Laplace's equation numerically.	3 min
Section 3	PINN model training: construction of the network, generation of interior and boundary collocation points, and definition of the PDE and boundary losses.	10 min
Section 4	Error analysis: comparison of the PINN prediction against the analytic solution, including an error map, cross-sections, and summary error statistics.	3 min
Section 5	Harmonic conjugate recovery: reconstructing v from the PINN's learned u field using the Cauchy-Riemann relations.	3 min
Section 6	Domain coloring: visualization of six complex maps using hue for argument and brightness for modulus.	3 min
Section 7	Fluid flow interpretation (bonus): reading u as a velocity potential and v as a stream function, and displaying the orthogonality of equipotential lines and streamlines.	2 min

8 Connections to the Complex Analysis Coursework

One of the explicit goals of this project was to make visible, in a single computational pipeline, how specific results from the complex analysis coursework are actually used. Table 5 reproduces this mapping, as laid out at the end of the notebook itself.

Table 5: Mapping between complex analysis coursework and project components.

Complex analysis concept	Realization in this project
Ch. 2 Cauchy-Riemann equations	Verified for the network's learned u , via the $u_x = v_y$ check in Section 5 of the notebook.
Ch. 2 Complex differentiation	<code>torch.autograd</code> acts as a computational stand-in for taking derivatives of an analytic function.
Ch. 4 Analytic functions	The harmonic-function property of $f(z) = z^2$ underlies the entire boundary value problem.
Ch. 5 Line integrals	The harmonic conjugate v is recovered by numerically integrating u_x along y (Section 5 of the notebook).
Conformal maps	Domain coloring (Section 6 of the notebook) makes the angle-preserving behavior of analytic maps visible directly.

9 Discussion

The result that stands out most from this project is not numerical accuracy by itself, since the chosen benchmark $u(x, y) = x^2 - y^2$ is a low-degree polynomial that is comparatively easy for a network to represent. What is more significant is the structure of the learning problem: the network never observes a single labeled value of u in the interior of the domain, and yet it is able to recover, to an L_2 error of about 2×10^{-4} , a function that is close to the unique harmonic extension of the prescribed boundary data. This is possible precisely because the differential equation itself, not interior data, supplies the missing information, which is the defining idea of physics-informed machine learning as introduced by Raissi, Perdikaris, and Karniadakis [5, 6, 7].

The project also illustrates, in a concrete and reproducible way, how several classical results from complex analysis interact with one another in a single computational example. The Cauchy-Riemann equations are not only used as a verification step for u and v in Section 3.1 of this report, but are reused as an active computational tool in Section 5 of the notebook, where they are applied to the network's learned u field in order to reconstruct the harmonic conjugate v without ever training a separate model for v . The domain coloring visualization in Section 6 of the notebook connects the same function $f(z) = z^2$, alongside several other standard examples, to the broader subject of conformal and complex-function visualization, while the fluid flow interpretation in Section 7 connects the harmonic pair (u, v) to two-dimensional potential flow, where u plays the role of velocity potential and v the role of stream function, and where the orthogonality of their level curves is itself a direct geometric consequence of the Cauchy-Riemann equations.

Taken together, the project is intended to show that an analytic function as simple as $f(z) = z^2$ already carries enough mathematical structure, through the Cauchy-Riemann equations and Laplace's equation, to support a complete small-scale scientific machine learning study: a well-posed boundary value problem, a verifiable ground truth, a trainable physics-informed model, a quantitative error analysis, and at least two independent physical or geometric interpretations of the resulting harmonic pair.

10 Future Directions

The notebook closes with a short list of possible extensions, included here for completeness, since they outline natural next steps beyond the scope of the current implementation.

1. **More general boundary conditions.** Replacing $u|_{\partial D} = x^2 - y^2$ with boundary data that is not the restriction of an analytic function's real part, to test the PINN on problems without a closed-form harmonic solution.
2. **Poisson's equation.** Extending $\nabla^2 u = 0$ to $\nabla^2 u = f$ for a prescribed source term f , modeling fields with sources or sinks rather than purely source-free harmonic fields.
3. **Three-dimensional PINNs.** Extending the input from (x, y) to (x, y, z) to solve the three-dimensional Laplace equation.
4. **Fourier feature embeddings.** Augmenting the input encoding to improve the network's ability to approximate higher-frequency target functions, which plain coordinate inputs combined with tanh activations tend to under-resolve.
5. **Residue theorem applications.** Using the residue theorem (covered in later chapters of the reference lecture notes) to verify, numerically, the value of certain definite integrals computed by contour integration.

11 Conclusion

This project was developed independently after completing a course in complex analysis, with the goal of extending material from that course, specifically analytic functions, the Cauchy-Riemann equations, harmonic functions, Laplace's equation, and conformal visualization, into a research-style artifact that could be computed, trained, and visualized rather than only studied analytically. The function $f(z) = z^2$ was used as a mathematical benchmark: its real part $u(x, y) = x^2 - y^2$ was shown directly to satisfy Laplace's equation, and a Physics-Informed Neural Network was then trained to learn this same harmonic function using only the PDE residual and the boundary condition, without access to any interior ground-truth data. The resulting model was evaluated through error analysis against the exact solution, used to recover the harmonic conjugate via the Cauchy-Riemann equations, visualized through domain coloring, and reinterpreted physically through the lens of two-dimensional potential flow.

Overall, the project is meant to demonstrate the ability to translate theoretical results in complex analysis into a working machine learning model and a complete scientific computing pipeline, and to serve as a concrete, self-contained piece of evidence of mathematical understanding, programming ability, and interdisciplinary research potential in the area of AI for Science.

Appendix: Project Summary for External Use

For reference, the following short descriptions of the project are suitable for inclusion in a CV, a LinkedIn profile, or similar external materials.

Project name. PINNs for Laplace's Equation: Complex Analysis and AI for Science

Project URL. <https://chelsietao.github.io/pinn-laplace-complex-analysis/>

Source code. <https://github.com/chelsietao/pinn-laplace-complex-analysis>

Description. Independently developed an AI for Science side project after completing a course in complex analysis, connecting analytic functions, the Cauchy-Riemann equations, harmonic functions, and Laplace's equation with Physics-Informed Neural Networks. Implemented a PyTorch-based PINN to solve the Dirichlet problem for Laplace's equation on a square domain, using $u(x, y) = x^2 - y^2$, the real part of $f(z) = z^2$, as the analytic benchmark. The project includes PDE residual training, boundary-condition loss, error analysis, harmonic conjugate recovery, domain coloring, and a physical interpretation through potential flow visualization.

References

- [1] Teh, Jyh-Haur. *Lecture Notes on Complex Analysis: Analytic Functions, Integrations and Singularities*. Department of Mathematics, National Tsing Hua University. Chapters referenced: Chapter 2 (Complex Differentiation), Chapter 4 (Analytic Functions), Chapter 5 (Line Integrals), Chapter 16 (Riemann Mapping Theorem / Conformal Maps).
- [2] Bak, J., and Newman, D. J. *Complex Analysis*. Springer.
- [3] Conway, J. B. *Functions of One Complex Variable*. Springer.
- [4] Rudin, W. *Real and Complex Analysis*. McGraw-Hill.

- [5] Raissi, M., Perdikaris, P., and Karniadakis, G. E. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations." *Journal of Computational Physics*, 378, 686–707, 2019.
- [6] Raissi, M., Perdikaris, P., and Karniadakis, G. E. "Physics Informed Deep Learning (Part I): Data-driven Solutions of Nonlinear Partial Differential Equations." arXiv:1711.10561, 2017.
- [7] Raissi, M., Perdikaris, P., and Karniadakis, G. E. "Physics Informed Deep Learning (Part II): Data-driven Discovery of Nonlinear Partial Differential Equations." arXiv:1711.10566, 2017.
- [8] PyTorch Documentation. *Automatic Differentiation Package – torch.autograd*. <https://docs.pytorch.org/docs/stable/autograd.html>
- [9] PyTorch Documentation. *torch.autograd.grad*. <https://docs.pytorch.org/docs/stable/generated/torch.autograd.grad.html>
- [10] NumPy Documentation. <https://numpy.org/doc/>
- [11] Matplotlib Documentation. <https://matplotlib.org/stable/index.html>